

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

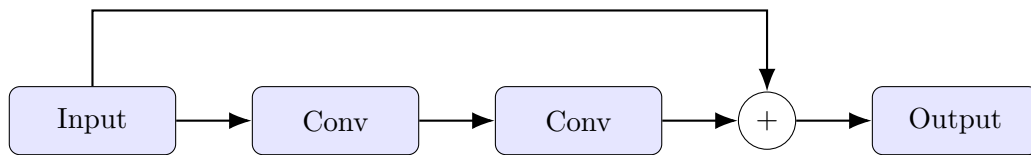


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

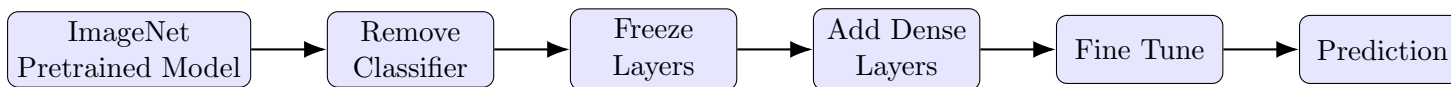


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Plots

15.1 Sample CIFAR-10 Images

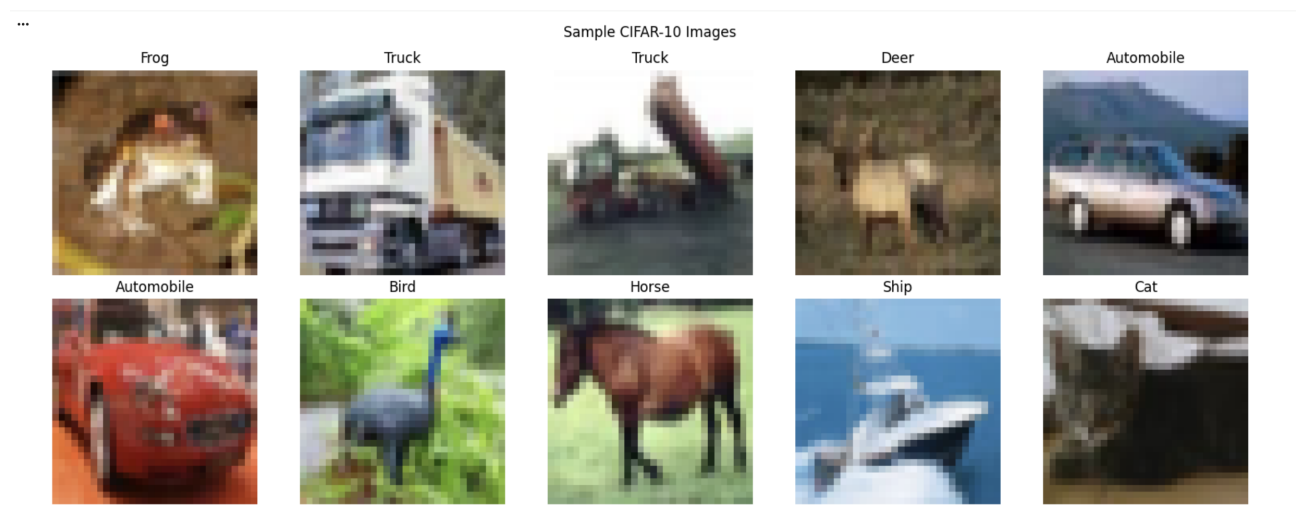


Figure 3: Sample CIFAR-10 Images

Inference: The figure shows ten sample images from the CIFAR-10 dataset belonging to different classes such as Frog, Truck, Deer, Automobile, Bird, Horse, Ship, and Cat. These images demonstrate the diversity of objects present in the dataset and confirm that the images are RGB images of size 32×32 pixels.

15.2 Training Accuracy vs Validation Accuracy

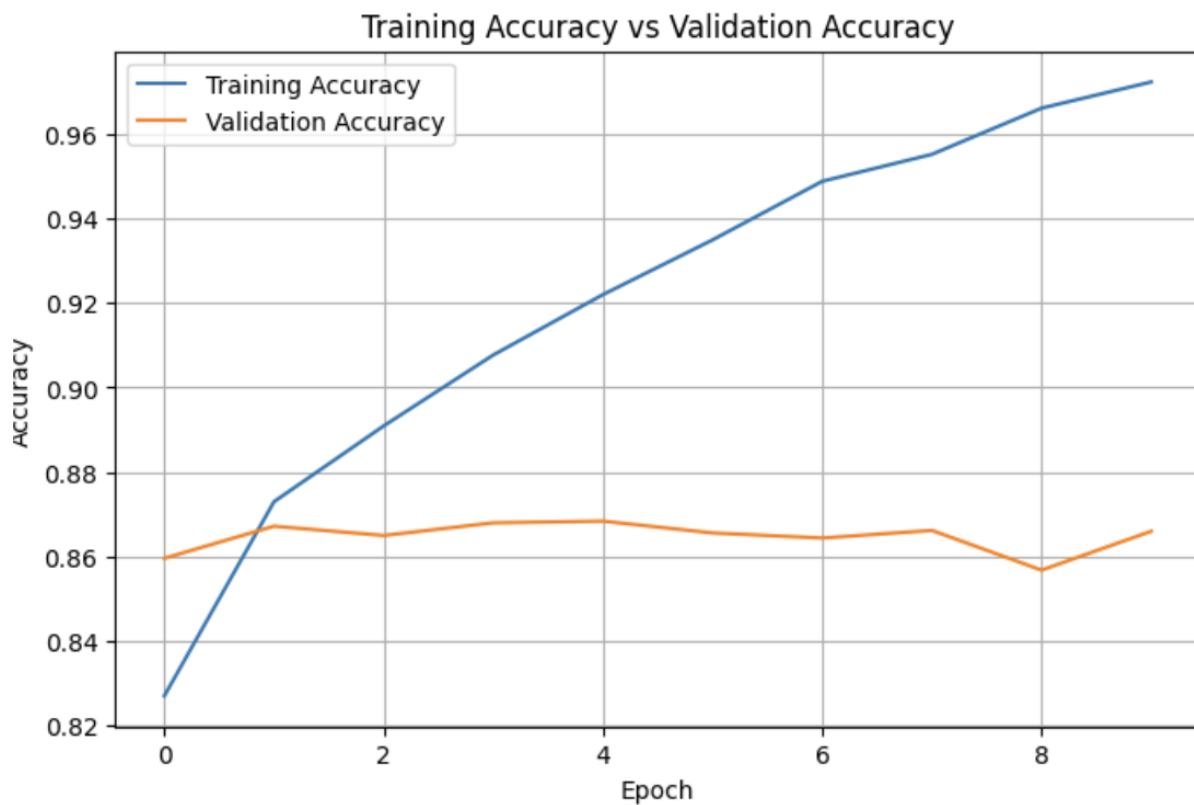


Figure 4: Training Accuracy vs Validation Accuracy

Inference: The training accuracy increases steadily from approximately 82.7% to 97.3% over the epochs, indicating that the model learns the training data effectively. However, validation accuracy remains around 86–87% and fluctuates slightly, indicating a growing gap between training and validation performance and possible overfitting in the later epochs.

15.3 Training Loss vs Validation Loss

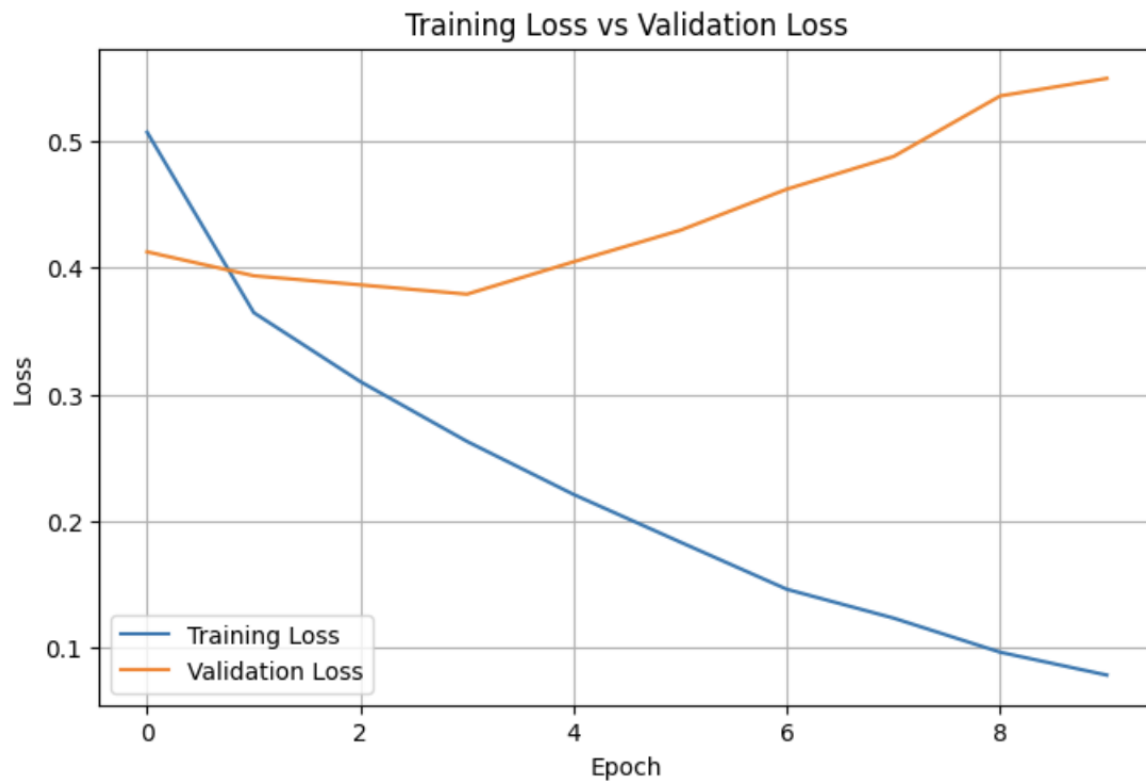


Figure 5: Training Loss vs Validation Loss

Inference: The training loss decreases steadily from approximately 0.51 to 0.08, indicating that the model is learning the training data effectively. The validation loss decreases initially and reaches its minimum around epoch 3, but then increases steadily, indicating overfitting as training progresses.

15.4 Fine-Tuning Loss

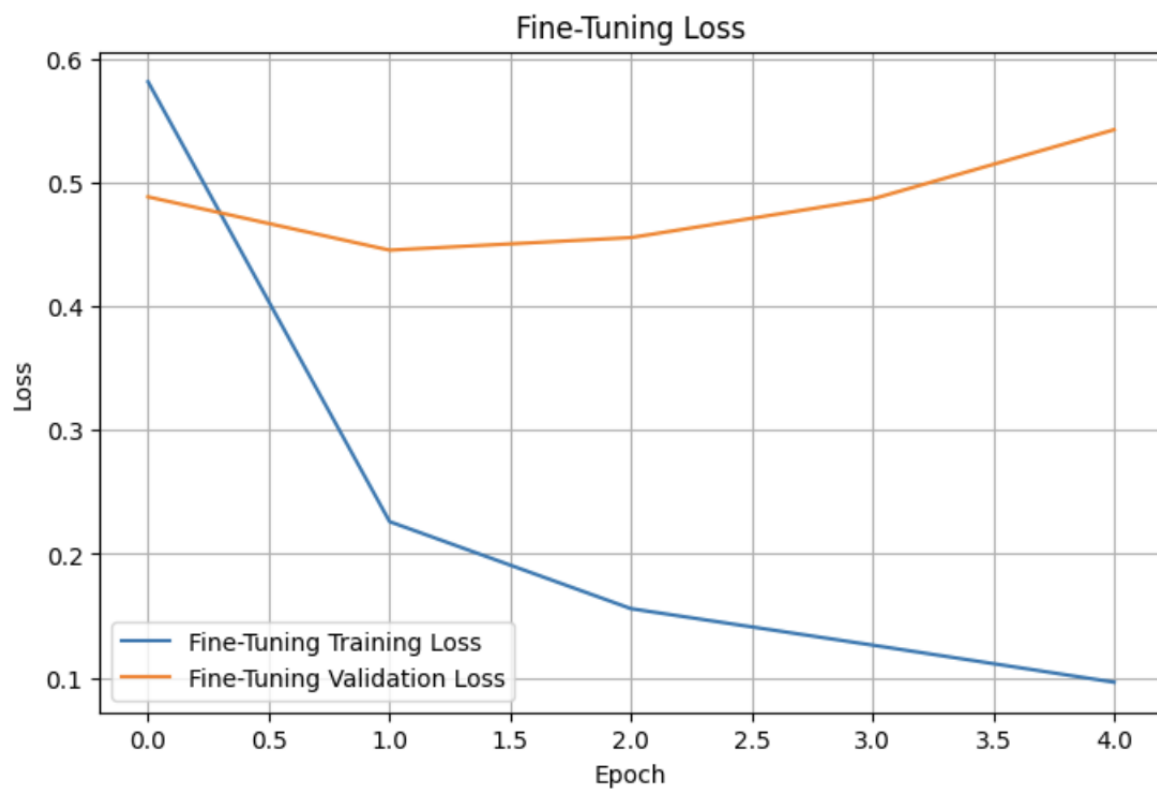


Figure 6: Fine-Tuning Loss

Inference: The fine-tuning training loss decreases consistently from approximately 0.58 to 0.10, indicating effective learning on the training data. The validation loss decreases initially and reaches its minimum around epoch 1, but then increases gradually, indicating mild overfitting during the later fine-tuning epochs.

15.5 Fine-Tuning Accuracy

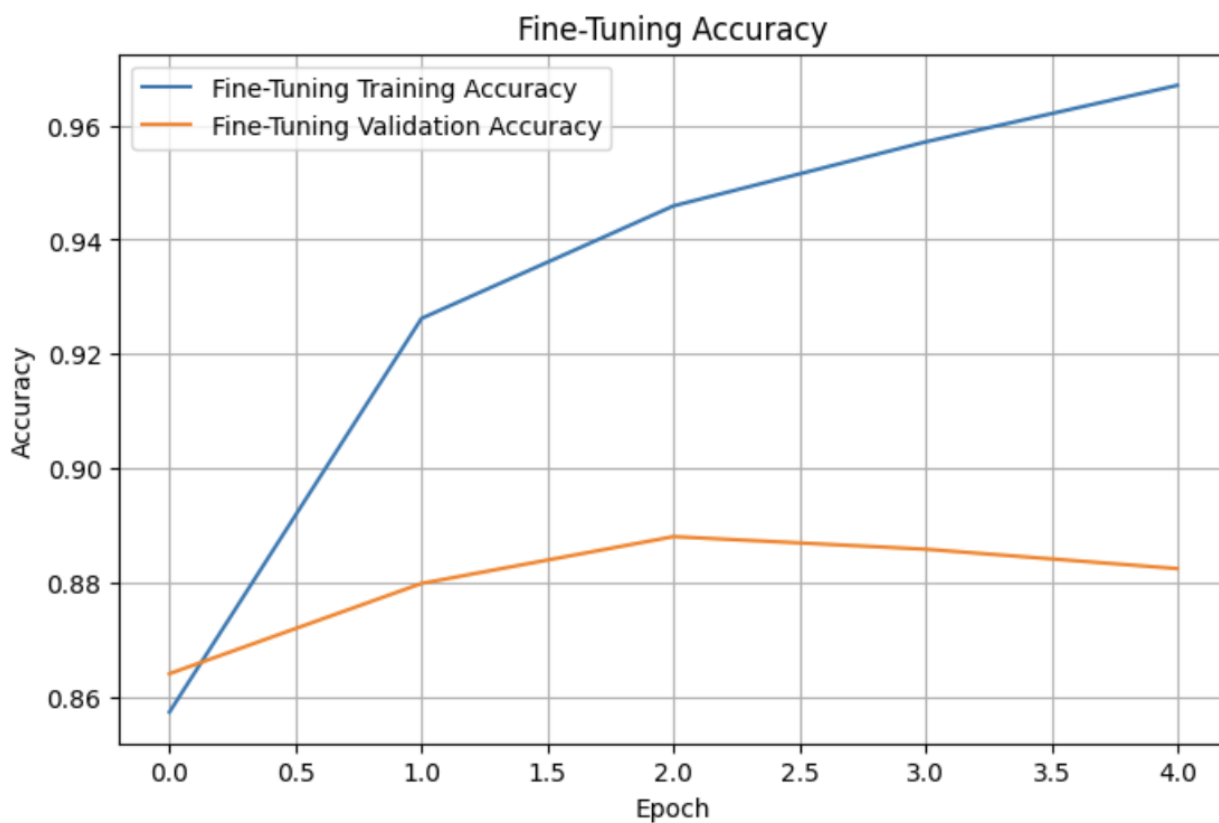


Figure 7: Fine-Tuning Accuracy

Inference: Fine-tuning increases the training accuracy from approximately 85.7% to 96.7%, showing that the pretrained model adapts effectively to the CIFAR-10 dataset. Validation accuracy improves initially and reaches approximately 88.8% around epoch 2, after which it decreases slightly, indicating mild overfitting in the later fine-tuning epochs.

15.6 Confusion Matrix

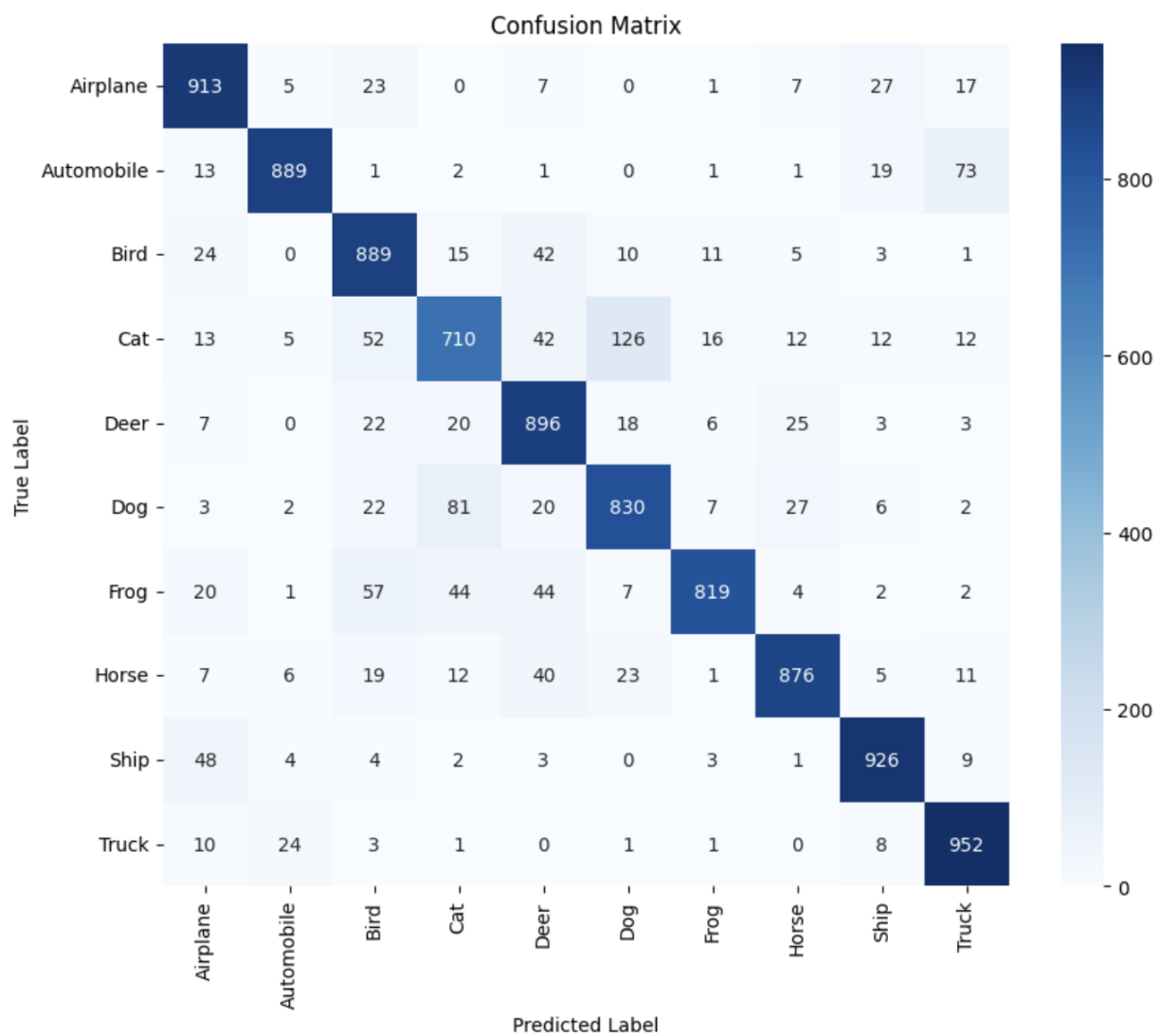


Figure 8: Confusion Matrix for CIFAR-10 Classification

Inference: The confusion matrix shows that most predictions lie along the diagonal, indicating that the model correctly classifies a large proportion of the CIFAR-10 test images. Some confusion is observed between visually similar classes, particularly Cat and Dog, Automobile and Truck, and Bird and Deer.

15.7 Misclassified CIFAR-10 Images

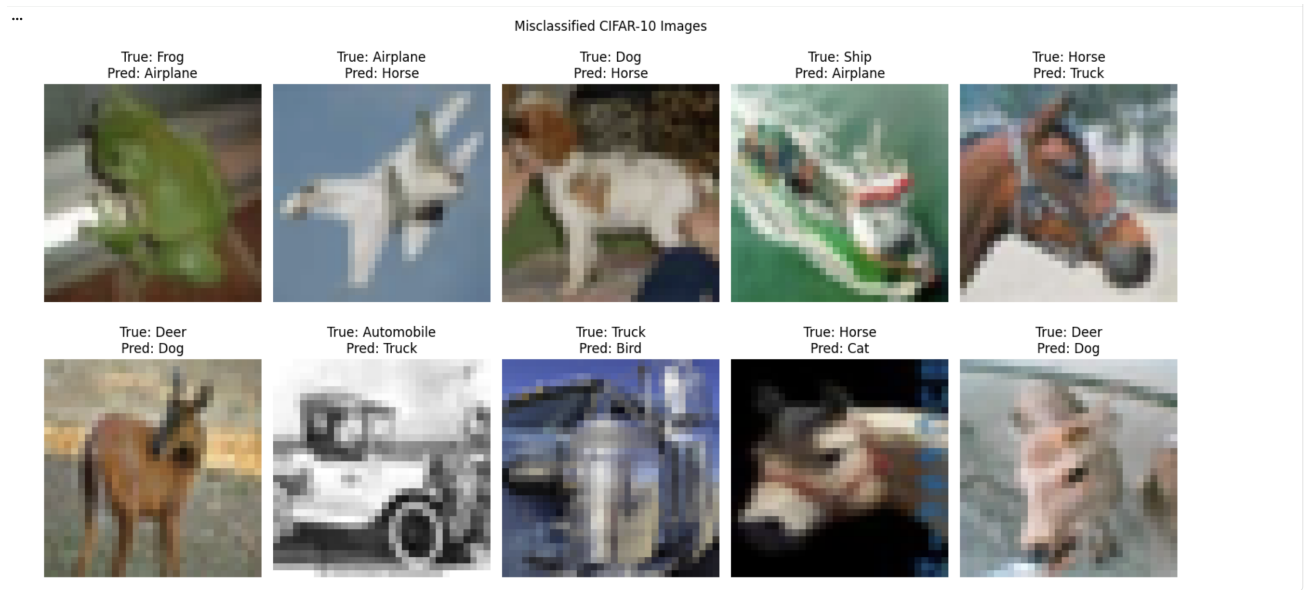


Figure 9: Misclassified CIFAR-10 Images

Inference: The figure shows examples where the model incorrectly classified CIFAR-10 images, including Frog as Airplane, Dog as Horse, and Automobile as Truck. These errors occur mainly between visually similar or low-resolution objects, highlighting the difficulty of distinguishing certain classes in the 32×32 CIFAR-10 images.

15.8 Hyperparameter Study

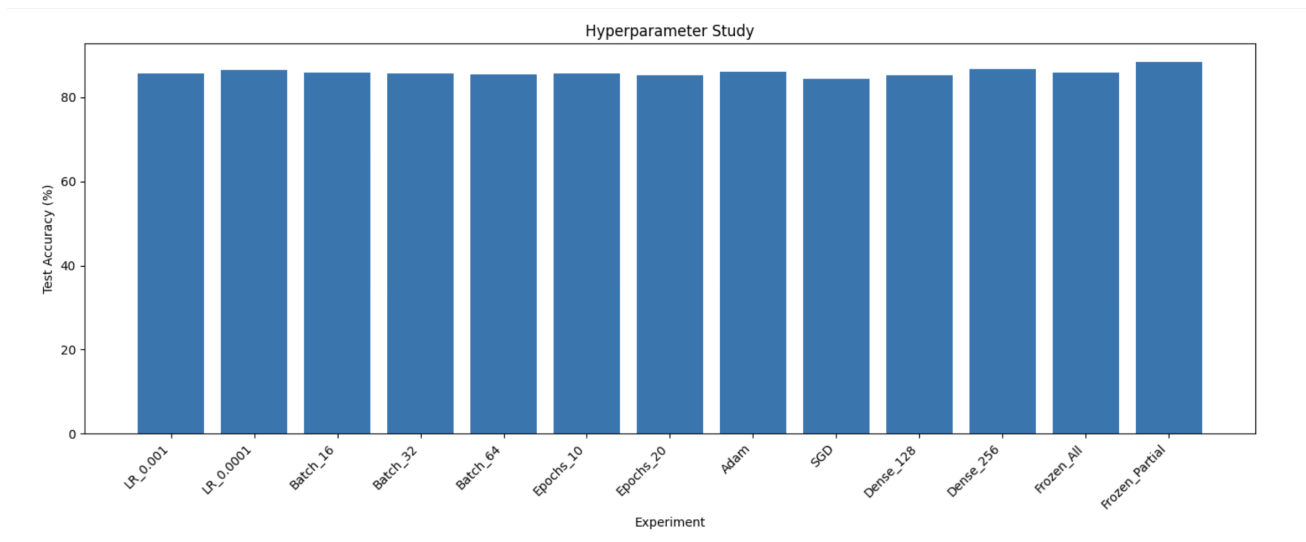


Figure 10: Hyperparameter Study

Inference: The hyperparameter study shows that the model performance varies slightly with changes in learning rate, batch size, epochs, optimizer, dense units, and the number of frozen layers. Among the tested configurations, partial freezing achieves the highest test accuracy, indicating that allowing some pretrained layers to adapt to the CIFAR-10 dataset improves performance.

15.9 Comparison of CNN Architectures

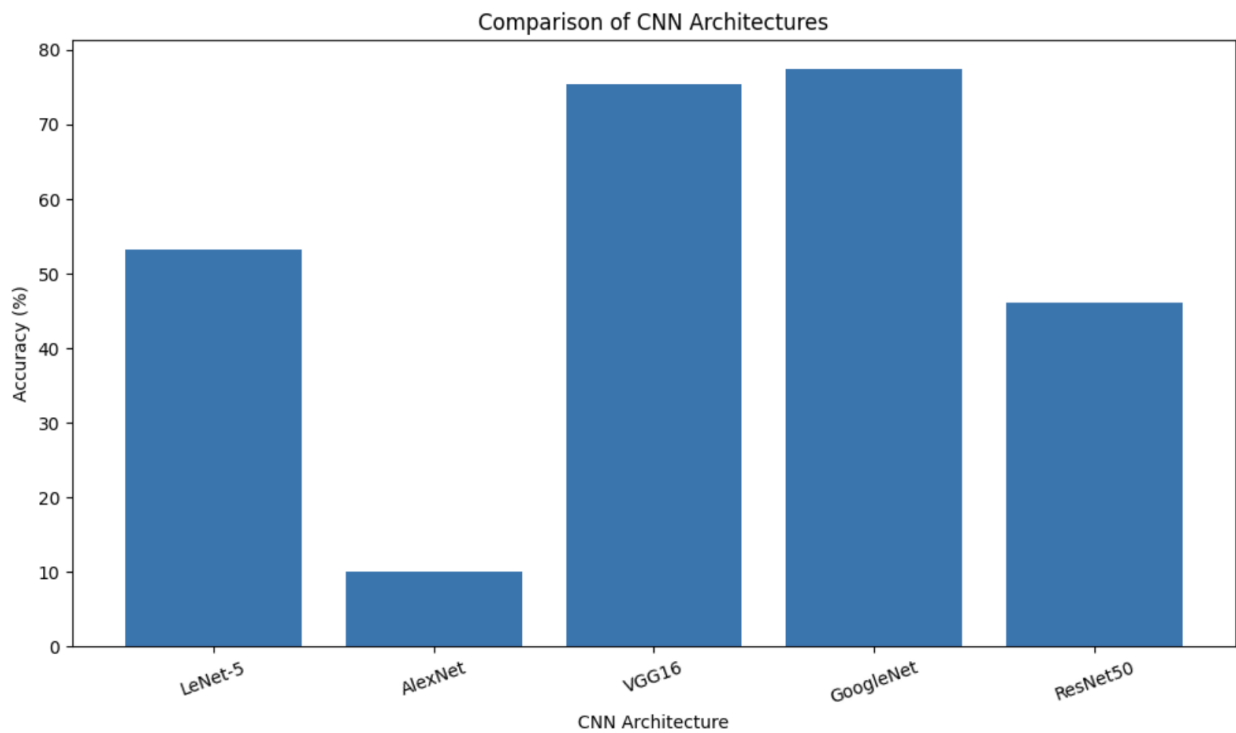


Figure 11: Comparison of CNN Architectures

Inference: The comparison shows that GoogleNet achieves the highest classification accuracy at approximately 77%, followed by VGG16 at approximately 75%. LeNet-5 and ResNet50 achieve moderate accuracy, while AlexNet records the lowest accuracy among the tested architectures.

15.10 Training Time Comparison of CNN Architectures

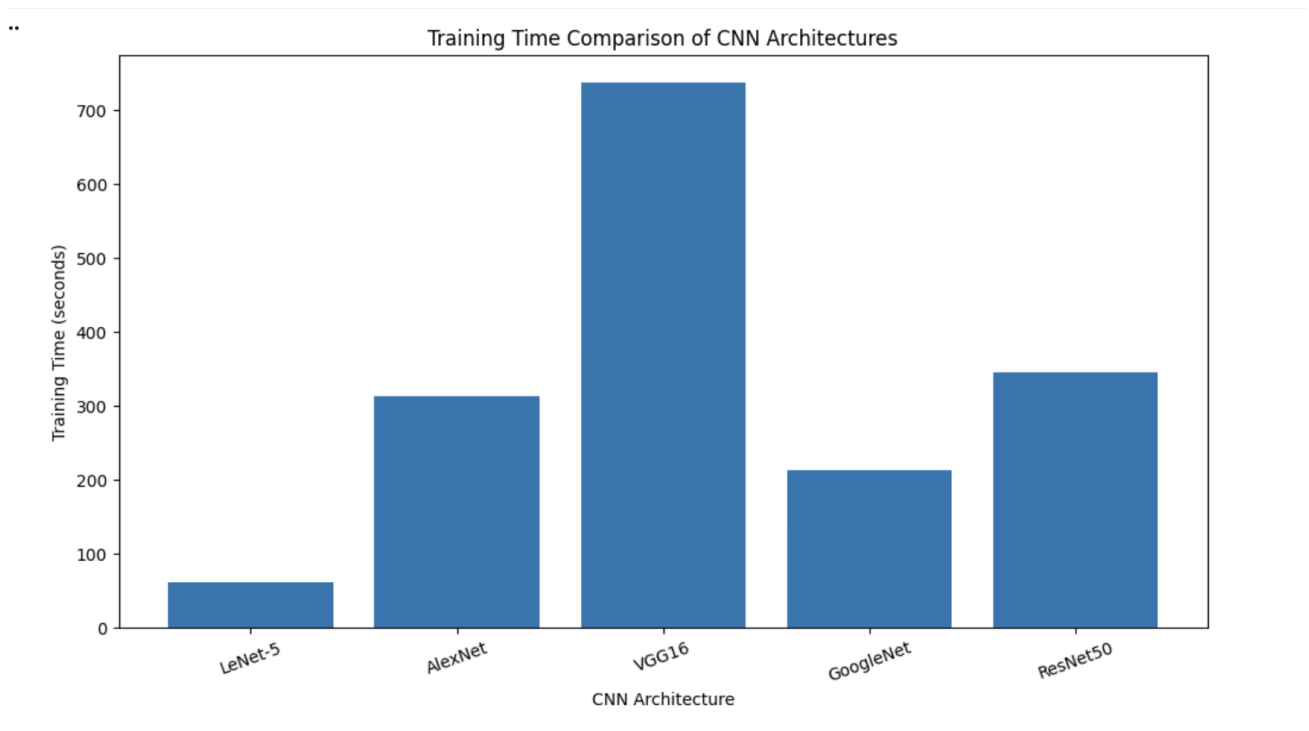


Figure 12: Training Time Comparison of CNN Architectures

Inference: VGG16 requires the highest training time at approximately 738 seconds, while LeNet-5 has the lowest training time at approximately 62 seconds. The results show that deeper and more computationally complex architectures generally require more time for training compared with simpler CNN architectures.

16. Results

16.1 Performance Metrics

Metric	Value
Training Accuracy	96.70%
Testing Accuracy	87.00%
Precision	0.8719
Recall	0.8700
F1-score	0.8695
Training Time	161.35 seconds
Total Parameters	2,423,242

16.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	83,126	53.16	62.33
AlexNet	36,925,322	10.00	313.78
VGG16	14,781,642	75.49	737.31
GoogLeNet	856,859	77.41	212.92
ResNet50	23,851,274	46.08	346.43

17. Discussion Questions

1. **Why is AlexNet considered a breakthrough in deep learning?**

AlexNet was a breakthrough because it significantly reduced the classification error in the 2012 ImageNet Challenge. It introduced ReLU activation, Dropout, GPU-based training, and data augmentation, which improved deep CNN performance.

2. **Why does VGG16 use only 3×3 convolution filters?**

VGG16 uses small 3×3 filters repeatedly to build a deeper network while effectively extracting features. This provides excellent feature extraction while maintaining a simple and uniform architecture.

3. **Explain the advantages of the Inception module.**

The Inception module performs multiple convolutions with different filter sizes in parallel, such as 1×1 , 3×3 , and 5×5 , along with max pooling. It enables multi-scale feature extraction, reduces computational complexity, and requires fewer parameters than VGG16.

4. **What is the purpose of residual learning?**

Residual learning helps solve the vanishing gradient problem in very deep CNNs. ResNet uses skip connections that allow gradients to flow directly through the network, enabling effective training of deep networks.

5. **Differentiate LeNet and ResNet.**

LeNet-5 is a small and simple CNN with two convolution layers and two average pooling layers, mainly designed for handwritten digit recognition. ResNet is a much deeper architecture that uses residual or skip connections to enable effective training of deep networks.

6. **What is Transfer Learning?**

Transfer learning uses a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet. The pretrained model is adapted to a new dataset by replacing its classifier, freezing layers, adding new layers, and optionally fine-tuning selected convolutional layers.

7. **Why is fine tuning required?**

Fine tuning allows selected layers of a pretrained model to adapt their learned features to the new dataset. It can improve classification performance while retaining the useful features learned from the original dataset.

8. **Explain the difference between dilated convolution and transpose convolution.**

Dilated convolution increases the receptive field by inserting gaps between kernel elements without increasing the number of parameters. Transpose convolution performs learnable upsampling and increases the spatial dimensions of feature maps.

9. **Why do pretrained models converge faster?**

Pretrained models have already learned general image features from a large dataset such as ImageNet. Therefore, they require less learning from scratch, resulting in faster convergence, reduced training time, and often better accuracy on small datasets.

10. **Compare the computational complexity of LeNet and ResNet.**

LeNet has a small number of parameters and a shallow architecture, making it computationally inexpensive and fast to train. ResNet is much deeper and requires substantially more computation, but its residual connections allow it to train very deep networks effectively. LeNet-5 has

approximately 60K parameters, whereas ResNet50 has approximately 25.6M parameters.

18. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

19. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>
9. Code file :- https://colab.research.google.com/drive/1Bg_p5zQ73fLyp4RqfscPHpaCqUF9r1v0?hl=en
10. Github Link:- <https://github.com/Asmita-306/DL-Lab-4>